

1.Title -

Automated Meeting Minutes Generation Using Natural Language Processing and Large Language Models

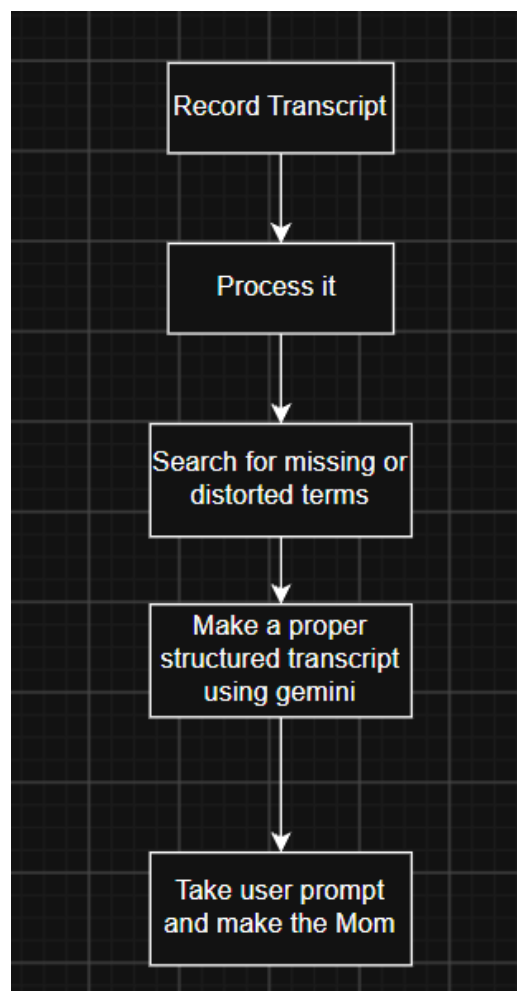
2.Abstract -

Meetings often contain valuable information in the form of discussions, decisions, action items, deadlines, risks, and follow-up tasks, but manually preparing Minutes of Meeting (MoM) is time-consuming and prone to missing important details. This project proposes an automated meeting-minutes generation system that converts raw meeting conversations or transcripts into structured and concise Minutes of Meeting. The system processes meeting dialogue containing interruptions, informal language, follow-up questions, ambiguous context, and domain-specific terminology, and extracts key information such as meeting summaries, decisions, action items, responsible participants, deadlines, risks or blockers, and next-meeting information. A structured training dataset containing realistic meeting scenarios from domains such as software development, marketing, finance, sales, research, security, human resources, and AI is used to train and evaluate the system. The generated summaries can be evaluated using automatic metrics such as ROUGE, along with human evaluation for relevance, completeness, and factual consistency. The proposed system aims to reduce the manual effort required for documentation while producing standardized and actionable meeting records that can support better follow-up and organizational productivity.

3.TechStack-

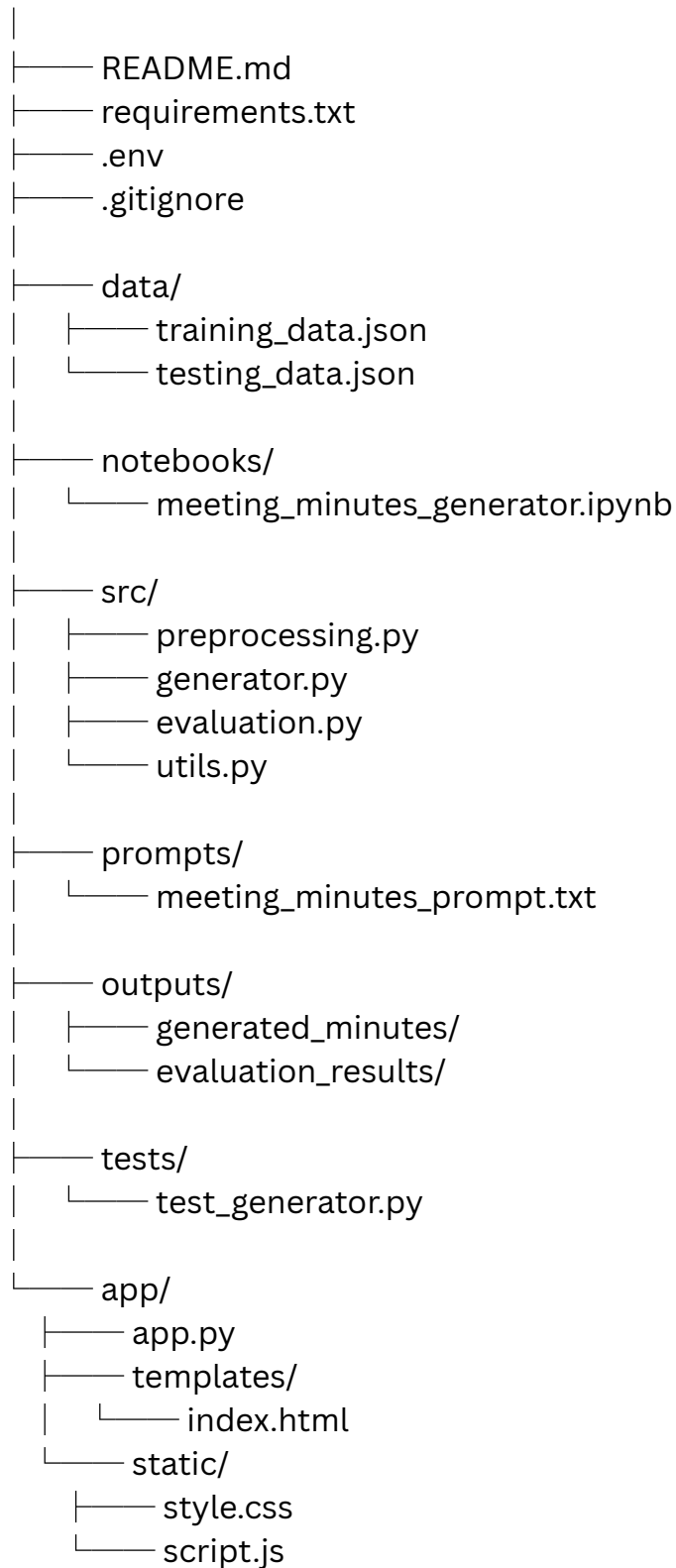
Python, Assembly U3.5pro, Gemini3.8 flash , Javascript , automatic email scheduling and calender scheduling (N8N) or any free tool , Supabase,Vercel , extension for gmeets and zoom.

4.Flow/SystemArchitecture-



5.Directory Structure-

Automated-MoM-Generator/



5.Testing and Performace-

Dataset is provided

6.Future Scope -

Further Enhancement

Easy

- Multiple export formats — Word (.docx) or PDF output for actually sending minutes to attendees, not just JSON/Markdown.
- Priority/urgency tagging — add a priority: Literal["low","medium","high"] field to ActionItem.
- Timestamps in output — if the transcript includes timestamps, carry them through so each topic/decision links back to "discussed at 14:32."
- Email-ready summary — a formatter that turns MeetingMinutes into a ready-to-send email body (subject line + body), since that's the actual end use case for most people.
- Multiple meeting types/templates — a standup transcript needs different fields (blockers, yesterday/today) than a client call (next steps, risks) — make the schema swappable per meeting type.

Medium

- Speaker-aware parsing — currently it treats the whole transcript as one blob; explicitly parsing "Name: text" lines lets the model reliably attribute decisions/action items to who actually said them, instead of guessing.
- Action item deduplication across meetings — if the same action item ("follow up with legal") appears in consecutive weekly meetings, detect it's a carryover rather than a new item.
- Sentiment/tone flagging — flag if a discussion got contentious or if a decision was made without consensus (useful for team leads reviewing minutes later).
- Calendar/task integration — push generated action items directly to Google Calendar, Asana, or Trello via their APIs instead of just printing them (this is where the LangChain MCP tool pattern would fit well).
- Follow-up reminder generator — auto-draft a "reminder" message for each action item nearing its deadline.

Harder

- Diarization from raw audio — right now it assumes a clean text transcript; students could add a speech-to-text + speaker-diarization step (e.g. Whisper + pyannote) so the input is an actual audio recording of the meeting.
- Chunking, done properly — the current chunking is naive (split by character count, then merge partial summaries). A better version would chunk by topic boundaries (detect topic shifts) rather than raw length, so a decision made right at a chunk boundary doesn't get split awkwardly.
- Multi-meeting trend analysis — across a series of meetings for the same project, track which action items keep slipping, which decisions get revisited, and surface that as a report.
- Accuracy validation — extractive grounding, similar to the code-explainer suggestion: have the model cite the exact line(s) supporting each decision/action item, so a human can spot-check instead of trusting it blindly.
- Real-time mode — instead of a full transcript after the fact, feed it a live streaming transcript and have it incrementally update minutes as the meeting happens.